

Employing Metaprogramming: A Macro-Driven Strategy to Overcome Compiler-Imposed Limitations on Specialization

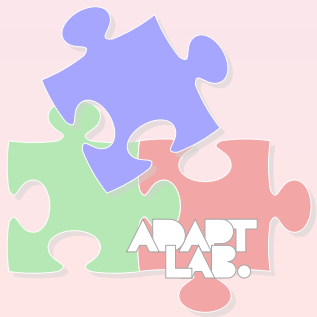
Davide Cerato
Id. Number: 984481

Università degli Studi di Milano
Computer Science Department
BSc in Computer Science

Advisor: Prof. Walter Cazzola
Co-Advisor: M.Sc. Federico Bruzzone

XX/02/2026

L-31 - Computer Science
Academic Year 2024-2025





Specialization
State of the Art and Limitations

Specialization

Problem Statement

Proposal

By Example: Implementing the Proposal

Implementation Details

Limitations

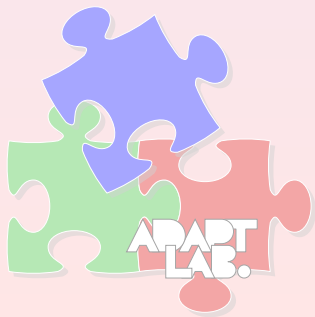
Evaluation

Conclusions

Systems programming aims to combine generic programming with type-specific optimizations.

Mechanisms vary widely, from mature and widely used systems to experimental features, with varying degrees of success.

In many modern ecosystems, specialization is often unstable, forcing a tradeoff between performance and compiler stability.



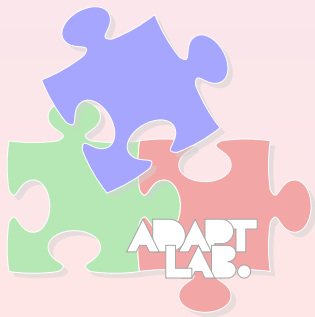


Problem Statement
What's happening in Rust?

In 2015, an RFC introduced specialization to Rust as an unstable feature confined to the Nightly Channel.

However, due to unresolved design issues and concerns about soundness of the system, specialization is still unstable.

Without stable specialization, developers often resort to code duplication or complex workarounds.



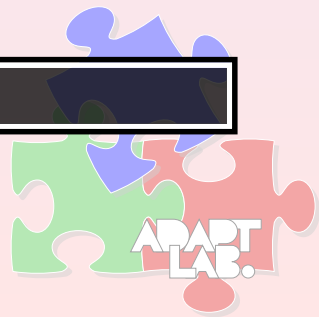


Problem Statement
Motivational Example

Consider implementing a trait for different types, with a specialized implementation for a specific type.

```
1  trait MyTrait { fn my_method(&self); }
2  impl<T> MyTrait for T {
3      fn my_method(&self) {
4          println!("Generic implementation");
5      }
6  }
7  impl MyTrait for i32 {
8      fn my_method(&self) {
9          println!("Specialized for i32");
10     }
11 }
12 fn main() {
13     10.my_method();   // should print "Specialized for i32"
14     10.5.my_method(); // should print "Generic implementation"
15 }
```

```
error[E0119]: conflicting implementations of trait MyTrait for type i32
```





Proposal
Macro-Based Specialization

Employ the powerful metaprogramming capabilities of Rust to build a macro library that simulates native specialization, with:

- Native-like look and feel
- Compile-time resolution
- Sound and stable
- Zero runtime overhead
- Powerful API

To do this, we use two macros: `#when` and `spec!`.

The approach can also be applied to other languages with powerful metaprogramming capabilities that lack native specialization support.

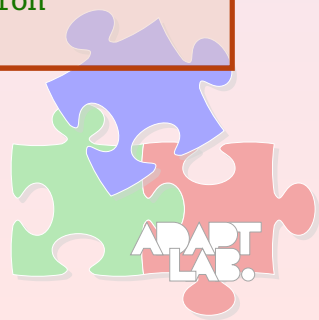




By Example: Implementing the Proposal
Motivational Example

To illustrate how the proposed approach works in practice, let's revisit the motivational example we presented earlier.

```
1  trait MyTrait { fn my_method(&self); }
2  impl<T> MyTrait for T {
3      fn my_method(&self) {
4          println!("Generic implementation");
5      }
6  }
7  #[when(T = i32)]
8  impl<T> MyTrait for T {
9      fn my_method(&self) {
10         println!("Specialized for i32");
11     }
12 }
13 fn main() {
14     spec! { 10.my_method(); i32 };       // prints "Specialized for i32"
15     spec! { 10.5.my_method(); f64 };     // prints "Generic implementation"
16 }
```



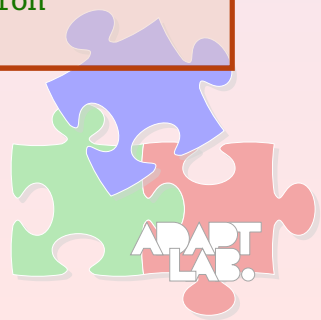


By Example: Implementing the Proposal

Motivational Example

To illustrate how the proposed approach works in practice, let's revisit the motivational example we presented earlier.

```
1  trait MyTrait { fn my_method(&self); }
2  impl<T> MyTrait for T {
3      fn my_method(&self) {
4          println!("Generic implementation");
5      }
6  }
7  trait MyTraitI32 { fn my_method(&self); }
8  impl MyTraitI32 for i32 {
9      fn my_method(&self) {
10         println!("Specialized for i32");
11     }
12 }
13 fn main() {
14     <i32 as MyTraitI32>::my_method(&10); // prints "Specialized for i32"
15     <f64 as MyTrait>::my_method(&10.5);  // prints "Generic implementation"
16 }
```





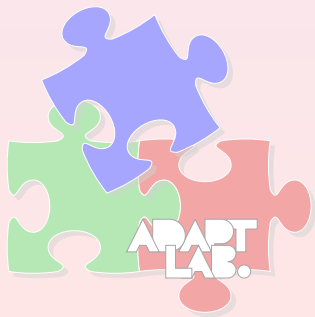
By Example: Implementing the Proposal
Supported Predicates and Forms

The following predicates are supported:

- `T = Type`
- `all(p1, p2, ...)`
- `not(p)`
- `T: Tr1 + Tr2`
- `any(p1, p2, ...)`

We also support the following implementations forms:

- `impl<T> Trait for T`
- `impl<T> Trait for Type<T>`
- `impl<T> Trait<T> for Type`





By Example: Implementing the Proposal
Lifetimes

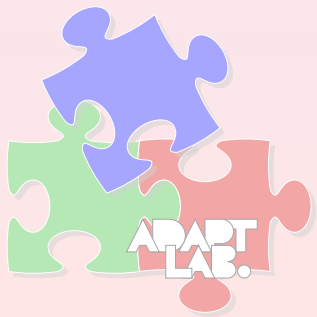
Lifetimes are one of the main reasons native specialization is still unstable.

Lifetimes bounds can be implicit or explicit.

In our belief, this is an architectural limitation of the compiler that can be overcome with proper re-design.

Our proposal treats lifetimes as first-class citizens:

- `T = &'a U`
- `T: 'a`

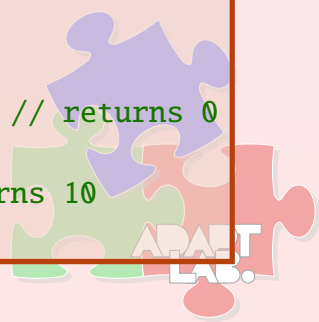




By Example: Implementing the Proposal
Higher-Ranked Polymorphism

Our macro system also supports Higher-Ranked Polymorphism.

```
1  struct S;
2  trait Trait<T, U, V> {
3      fn f(&self, t: T, u: U) -> V;
4  }
5  impl<T, U, V: Default> Trait<T, U, V> for S {
6      fn f(&self, t: T, u: U) -> V {
7          V::default()
8      }
9  }
10 #[when(all(
11     T = &str, T: 'b,
12     U = for<'a> fn(T, &'a i32) -> V
13 ))]
14 impl<'b, T, U, V> Trait<T, U, V> for S {
15     fn f(&self, t: T, u: U) -> V {
16         u(t, &7)
17     }
18 }
19 fn main() {
20     let s = S;
21     let lambda = |s: &str, n: &i32| s.len() as u32 + *n as u32;
23     let x: i32 = spec! { s.f("abc", 2); S; [&str, i32]; i32: Default }; // returns 0
24     let y: u32 = spec! { s.f("abc", lambda); S;
25       [&str, for<'a> fn(&str, &'a i32) -> u32]; u32: Default }; // returns 10
26 }
```

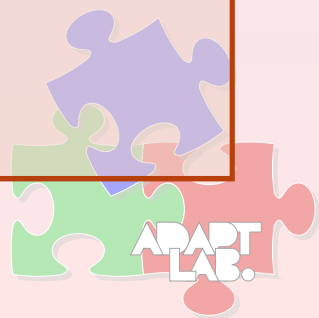




By Example: Implementing the Proposal
Higher-Ranked Polymorphism

Our macro system also supports Higher-Ranked Polymorphism.

```
1  struct S;
2  trait Trait<T, U, V> {
3      fn f(&self, t: T, u: U) -> V;
4  }
5  impl<T, U, V: Default> Trait<T, U, V> for S {
6      fn f(&self, t: T, u: U) -> V {
7          V::default()
8      }
9  }
10 trait TraitFn<'b, V> {
11     fn f(&self, t: &'b str, u: for<'a> fn(&'b str, &'a i32) -> V) -> V;
12 }
13 impl<'b, V> TraitFn<'b, V> for S {
14     fn f(&self, t: &'b str, u: for<'a> fn(&'b str, &'a i32) -> V) -> V {
15         u(t, &7)
16     }
17 }
18 fn main() {
19     let s = S;
20     let lambda = |s: &str, n: &i32| s.len() as u32 + *n as u32;
22     let x: i32 = <S as Trait<_, _, _>>::f(&s, "abc", 2);  // returns 0
23     let y: u32 = <S as TraitFn<_>>::f(&s, "abc", lambda); // returns 10
24 }
```





Implementation Details
Handling Conditional Implementations

Conditions defined in the `when` macro are normalized into DNF, flattening when possible, and then each disjunct condition is treated as a separate implementation, e.g.:

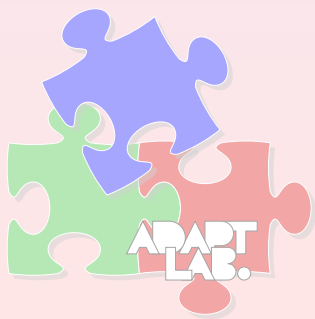
```
all(T = Vec<U>, any(U = i32, U = f64)) → any(T = Vec<i32>, T = Vec<f64>)
```

A partial order defined over the predicates is used to determine the specificity of each implementation, e.g.:

```
T = &i32 > T = &U > T: Copy
```

We need a way to collect all the traits defined in the crate and to handle the order of their implementations during method resolution. In Rust, this is done with Build Scripts, e.g.:

```
mono_spec::handle_order();
```





Limitations

Specialization

Problem Statement

Proposal

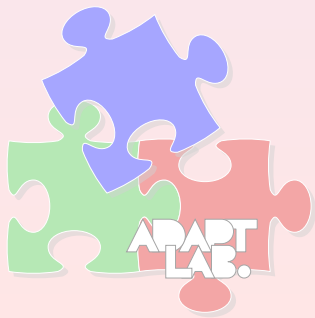
By Example: Implementing the Proposal

Implementation Details

Limitations

Evaluation

Conclusions





Evaluation
Approach

Specialization

Problem Statement

Proposal

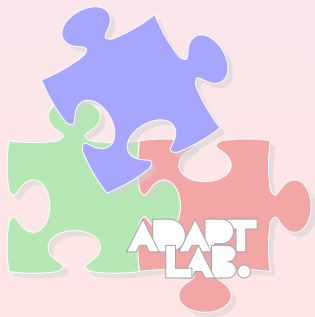
By Example: Implementing the Proposal

Implementation Details

Limitations

Evaluation

Conclusions





Evaluation
Results

Specialization

Problem Statement

Proposal

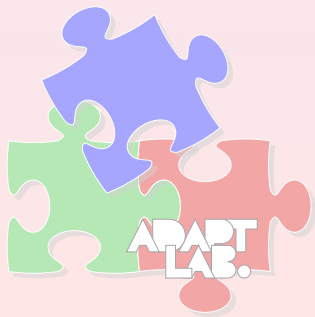
By Example: Implementing the Proposal

Implementation Details

Limitations

Evaluation

Conclusions





Conclusions

Interesting results:

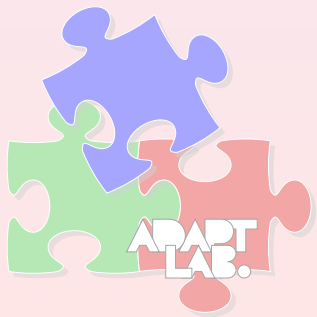
- We are writing an article (The Soul of Meta-Monomorphizing Specializations) to be submitted to ECOOP.

Interesting twist:

- xxx

Future work:

- xxx





Thanks for your attention!

